

Mike

Referensi Library & Command

15 library • 255 fungsi • seluruh keyword & command

Dokumen ini adalah referensi teknis lengkap bahasa Mike: seluruh command bahasa (keyword, builtin, perintah CLI) dan tiap fungsi dari 15 library standar — lengkap dengan parameter dan deskripsi singkat. Signature diambil langsung dari kode library, jadi akurat terhadap implementasi.

Bagian A — Command Bahasa

Struktur program

```
start module; <nama>
  lib input/output      # memuat library
  func main()           # titik masuk eksekusi
  ...
end func
end modul
```

Keyword

Keyword	Fungsi
start module; / end modul	membuka & menutup module
lib <nama>	memuat sebuah library
func / end func	mendefinisikan fungsi
return	mengembalikan nilai dari fungsi
var	mendeklarasikan variabel
if / else if / else / end if	percabangan
while / end while	perulangan berkondisi
for <i> = <a> to / end for	perulangan (inklusif kedua ujung)
print	cetak angka atau string + newline
input "prompt"	baca satu angka dari keyboard (ekspresi)
asm "..."	sisipkan instruksi assembly mentah
array(n)	alokasikan array n double
a[i]	akses/tulis elemen array
#	komentar hingga akhir baris

Operator

Kategori	Operator
Aritmetika	+ - * / %
Perbandingan	== != < <= > >=
Logika	and or not
Unary	-x (negasi) not x

Builtin matematika (selalu tersedia)

Fungsi ini tertanam di compiler, tak perlu library:

```
sqrt exp ln log10 sin cos tan floor ceil abs pow
```

Builtin I/O & data

Command	Keterangan
print <eksp>	cetak nilai numerik + newline

<code>print "teks"</code>	cetak string + newline
<code>input "prompt"</code>	baca satu angka dari stdin
<code>csv_load("file.csv", arr)</code>	muat semua angka CSV ke arr; kembalikan jumlah
<code>csv_rows("file.csv")</code>	hitung jumlah angka dalam CSV
<code>array(n)</code>	alokasikan n double; kembalikan handle
<code>random()</code>	double seragam [0,1)
<code>seed(n)</code>	seed generator acak
<code>rand_range(lo, hi)</code>	double seragam [lo, hi)
<code>gauss(mu, sd)</code>	sampel normal (Box-Muller)

Command CLI

Perintah	Aksi
<code>mike program.mik</code>	kompilasi → <code>./program</code> (executable native)
<code>mike -S program.mik</code>	hasilkan <code>program.s</code> (assembly) lalu berhenti
<code>./install.sh</code>	pasang mike per-user (<code>~/local/bin</code> , <code>~/mike/lib</code>)
<code>sudo ./install.sh --system</code>	pasang sistem (<code>/usr/local/bin</code> , <code>/usr/local/share</code>)
<code>./uninstall.sh</code>	hapus instalasi per-user

Bagian B — Referensi 15 Library

Setiap fungsi ditampilkan dengan parameter dan deskripsi singkat. Semua nilai bertipe double.

lib math

Konstanta & pembantu matematika — 15 fungsi

Fungsi	Parameter	Deskripsi
pi	—	
e	—	
tau	—	
sq	x	x^2 and x^3 without calling pow
cube	x	
sign	x	sign: -1, 0, or 1
min2	a, b	min / max of two values
max2	a, b	
clamp	x, lo, hi	clamp x into [lo, hi]
lerp	a, b, t	linear interpolation
rad	deg	degrees <-> radians
deg	rad	
round	x	round to nearest integer (via floor($x + 0.5$))
fact	n	factorial (iterative)
ipow	base, n	integer power by repeated multiply (works for negative exponents too)

lib stats

Statistika deskriptif, persentil, korelasi — 15 fungsi

Fungsi	Parameter	Deskripsi
sum	a, n	
mean	a, n	
variance	a, n	
var_sample	a, n	sample variance (divide by n-1)
stdev	a, n	
stdev_sample	a, n	
amin	a, n	
amax	a, n	
range	a, n	
sort	a, n	in-place insertion sort (ascending)
percentile	a, n, p	percentile p in [0,1] on a SORTED array (linear interpolation)
median	a, n	
skewness	a, n	skewness (population)
correlation	x, y, n	Pearson correlation between two arrays
covariance	x, y, n	covariance (population)

lib matrix

Matriks datar row-major — 10 fungsi

Fungsi	Parameter	Deskripsi
mget	m, cols, i, j	element access: get m[i][j]
mset	m, cols, i, j, v	element set: m[i][j] = v (returns v)
mfill	m, r, c, val	fill an r x c matrix with a constant
identity	m, n	identity into an n x n matrix
transpose	a, ra, ca, b	transpose A (ra x ca) into B (ca x ra)
matmul	a, ra, ca, b, cb, c	matrix multiply: A(ra x ca) * B(ca x cb) -> C(ra x cb)
matvec	a, ra, ca, v, out	matrix-vector product: A(ra x ca) * v(ca) -> out(ra)
trace	m, n	trace of an n x n matrix
det2	m	determinant of a 2x2 matrix
det3	m	determinant of a 3x3 matrix (Sarrus)

lib regression

Regresi kuadrat terkecil (least squares) — 5 fungsi

Fungsi	Parameter	Deskripsi
linreg	x, y, n, out	Simple linear regression $y = a + b \cdot x$ on n points. Writes: out[0] = intercept a, out[1] = slope b, out[2] = R^2 .
predict	a, b, x	Predict y from a fitted (a, b): $y = a + b \cdot x$
mse	x, y, n, a, b	Mean squared error of a linear fit over the data
rmse	x, y, n, a, b	Root mean squared error
expfit	x, y, n, out	Exponential fit $y = A \cdot \exp(B \cdot x)$ via linear regression on $\ln(y)$. Writes out[0] = A, out[1] = B, out[2] = R^2 (in log space). Requires all $y > 0$.

lib geometry

Vektor & geometri 2D/3D — 13 fungsi

Fungsi	Parameter	Deskripsi
dist2	x1, y1, x2, y2	
dot2	ax, ay, bx, by	
mag2	x, y	
cross2	ax, ay, bx, by	2D cross product (scalar): $ax \cdot by - ay \cdot bx$
cos_angle2	ax, ay, bx, by	angle of vector (x,y) from +x axis, using tan/cos identities. returns radians via atan approximation: $\text{angle} = \text{atan2}$ built from cos/sin. (Mike has sin/cos/tan; for atan we use a series-free trick with the provided builtins by normalizing — here we give the cosine of the angle.)
dist3	x1, y1, z1, x2, y2, z2	
dot3	ax, ay, az, bx, by, bz	
mag3	x, y, z	
cross3	ax, ay, az, bx, by, bz, out	3D cross product -> writes result into out[0..2]
tri_area	x1, y1, x2, y2, x3, y3	triangle area from three 2D points (shoelace)
poly_area	x, y, n	area of a polygon given arrays of x and y coords (shoelace), n vertices
circle_area	r	circle area / circumference

Fungsi	Parameter	Deskripsi
circle_circ	r	

lib finance

Matematika keuangan dengan parameter tunable — 27 fungsi

Fungsi	Parameter	Deskripsi
growth	r, n	$(1+r)^n$ via pow (n may be fractional for compounding conversions)
per_period	annual_rate, periods_per_year	convert an annual rate to a per-period rate for `m` periods/year e.g. per_period(0.12, 12) -> 0.01 monthly
fv_lump	pv, rate, nper	Future value of a present lump sum: $fv = pv \cdot (1+r)^n$
pv_lump	fv, rate, nper	Present value of a future lump sum: $pv = fv / (1+r)^n$
fv_annuity	pmt, rate, nper, type	Future value of a stream of `pmt` payments. type: 0 end-of-period, 1 beginning-of-period
pv_annuity	pmt, rate, nper, type	Present value of a stream of `pmt` payments.
pmt	pv, rate, nper, fv, type	Level payment for a loan of `pv` (like Excel PMT). Returns a NEGATIVE number (cash out) when pv is positive, by convention. type: 0 end-of-period, 1 beginning-of-period
pv_full	pmt_amt, rate, nper, fv, type	Present value of (loan payment stream + a future balloon fv).
nper	pv, rate, pmt_amt	Number of periods to pay off `pv` with payment `pmt` (both signs matter). $nper = \ln((pmt) / (pmt + pv \cdot rate)) / \ln(1+rate)$ [end-of-period]
npv	rate, cf, n	Net present value of cash flows $cf[0..n-1]$ discounted at `rate`. By convention $cf[0]$ is period-0 (usually the initial outflow, negative) and is NOT discounted. $cf[i]$ is discounted by $(1+rate)^i$.
irr	cf, n, lo, hi, iters	Internal rate of return via bisection on $npv() = 0$. Searches between `lo` and `hi` (e.g. -0.9 and 1.0) with `iters` steps. Returns the rate where NPV crosses zero. Robust for conventional flows.
ear	nominal, m	Effective annual rate from a nominal rate compounded m times/year. $ear(0.12, 12) \rightarrow 0.126825$ (12% nominal, monthly compounding)
nominal_from_ear	effective, m	Nominal rate (compounded m/year) that gives a target effective rate.
ear_continuous	nominal	Continuous-compounding effective rate: $e^r - 1$
cagr	begin_value, end_value, years	Compound annual growth rate over `years`.
rule_of_72	rate_percent	Rule of 72: approx years to double at rate r (as a percent, e.g. 6.0)
interest_payment	pv, rate, nper, k	Interest portion of payment #k (1-based) on a loan, end-of-period.
principal_payment	pv, rate, nper, k	Principal portion of payment #k.
balance_after	pv, rate, nper, k	Remaining balance after `k` payments.
dep_straight	cost, salvage, life	Straight-line depreciation per period.
dep_declining	cost, salvage, life, factor, k	Declining-balance depreciation expense in year `k` (1-based). `factor` tunes the method: 2.0 = double-declining, 1.5 = 150% DB, etc. Never depreciates below salvage.
bond_price	face, coupon_rate, ytm, years, freq	Price of a coupon bond. face: face / par value (e.g. 1000) coupon_rate: annual coupon rate (e.g. 0.05) ytm: yield to maturity (annual) years: years to maturity freq: coupon payments per year (1 annual, 2 semiannual)
current_yield	face, coupon_rate, price	Current yield = annual coupon / current price.
ytm_approx	face, coupon_rate, price, years	Approximate YTM (fast closed-form estimate; exact needs iteration). $approx = [C + (F - P)/n] / [(F + P)/2]$

Fungsi	Parameter	Deskripsi
hpr	begin_price, end_price, income	Simple holding-period return.
sharpe	mean_return, risk_free, stdev_return	Sharpe ratio given a mean return, risk-free rate, and stdev of returns.
real_return	nominal, inflation	Real return adjusted for inflation (Fisher equation).

lib science

Konstanta fisika & konversi satuan — 35 fungsi

Fungsi	Parameter	Deskripsi
c	—	
g_earth	—	
avogadro	—	
R_gas	—	
boltzmann	—	
planck	—	
elementary_charge	—	
stefan_boltzmann	—	
gravitation	—	
atm_pressure	—	
c_to_f	celsius	
f_to_c	fahrenheit	
c_to_k	celsius	
k_to_c	kelvin	
f_to_k	fahrenheit	
psi_to_pa	psi	
pa_to_psi	pa	
psi_to_bar	psi	
bar_to_psi	bar	
bar_to_pa	bar	
atm_to_psi	atm	
ft_to_m	ft	
m_to_ft	m	
inch_to_cm	inch	
mile_to_km	mile	
bbl_to_m3	bbl	
m3_to_bbl	m3	
bbl_to_ft3	bbl	
gal_to_liter	gal	
lb_to_kg	lb	
kg_to_lb	kg	
hp_to_watt	hp	
btu_to_joule	btu	

Fungsi	Parameter	Deskripsi
ideal_gas_pressure	n, temp_k, volume	---- ideal gas law helper: $PV = nRT$ ---- solve for pressure given n (mol), T (K), V (m^3)
ideal_gas_moles	pressure, volume, temp_k	solve for moles given P (Pa), V (m^3), T (K)

lib numeric

Metode numerik (root, integral, ODE) — 11 fungsi

Fungsi	Parameter	Deskripsi
bisection	a, b, iters	---- root finding (bisection) on user function f(x) ---- Requires: a function `f(x)` defined in the caller's program. Finds a root in [a,b] where f(a) and f(b) have opposite signs.
newton	x0, iters	---- Newton-Raphson on user function f(x) ---- Uses numerical derivative, so you only need to provide `f`.
derivative	x, h	---- numerical derivative of f at x (central difference) ----
derivative2	x, h	second derivative
integrate_trap	a, b, n	trapezoidal rule with n subintervals
integrate_simpson	a, b, n	Simpson's rule with n subintervals (n should be even)
integrate_data	y, n, dx	---- integration of SAMPLED data (arrays) ---- trapezoidal integration of y[] sampled at uniform spacing dx
integrate_xy	x, y, n	trapezoidal integration of y[] sampled at (possibly uneven) x[]
interp_linear	x, y, n, xq	linear interpolation: value at xq given sorted x[], y[]
ode_euler	x0, y0, x_end, steps	Euler's method
ode_rk4	x0, y0, x_end, steps	4th-order Runge-Kutta (much more accurate)

lib linalg

Aljabar linear umum $n \times n$ — 6 fungsi

Fungsi	Parameter	Deskripsi
get	m, n, i, j	element access on an n-column flat matrix
set	m, n, i, j, v	
solve	a, b, x, n	---- solve $Ax = b$ for an $n \times n$ system (Gaussian elimination with partial pivoting). A is $n \times n$ flat, b is length n. Solution is written into x. A and b are modified. ----
determinant	src, n	---- determinant of an $n \times n$ matrix via LU-style elimination. Works on a copy so the input is preserved. ----
inverse	src, inv, n	---- inverse of an $n \times n$ matrix via Gauss-Jordan. src preserved; result written into `inv` ($n \times n$ flat). ----
multireg	x, y, rows, cols, beta	---- multiple linear regression via normal equations. X is (rows x cols) flat, y is length rows. Solves $(X^T X) \beta = X^T y$. Writes beta (length cols). Include a column of 1s in X for an intercept. ----

lib complex

Bilangan kompleks (pasangan re, im) — 12 fungsi

Fungsi	Parameter	Deskripsi
c_add	ar, ai, br, bi, out	
c_sub	ar, ai, br, bi, out	
c_mul	ar, ai, br, bi, out	$(ar+ai*i)(br+bi*i) = (ar*br - ai*bi) + (ar*bi + ai*br) i$

Fungsi	Parameter	Deskripsi
c_div	ar, ai, br, bi, out	division: multiply by conjugate of denominator
c_abs	re, im	magnitude $ z = \sqrt{re^2 + im^2}$
c_abs2	re, im	squared magnitude $ z ^2$ (probability in quantum)
c_arg_approx	re, im	argument (phase angle) in radians, via a series-free construction: we use the identity atan2 through sign handling. Mike lacks atan2 , so we approximate using the relation with the unit circle: for most quantum work the phase is known analytically, but we provide a coarse arg via a Taylor-ish fallback for $ im/re $. For robust phase, prefer tracking phase analytically.
c_atan	t	atan approximation (good for $ t \leq 1$; uses reduction for $ t > 1$)
c_conj	re, im, out	
c_exp	re, im, out	complex exponential: $e^{(re + im*i)} = e^{re} * (\cos(im) + i \sin(im))$
c_polar	r, theta, out	from polar: $r * (\cos(theta) + i \sin(theta))$
c_expi	theta, out	$e^{(i*theta)}$ — unit complex, extremely common in quantum phases

lib quantum

Simulator komputasi kuantum state-vector — 13 fungsi

Fungsi	Parameter	Deskripsi
pow2	n	2^n (integer power of two)
qinit	state, n	initialize an n-qubit register to $ 00\dots0\rangle$ state must be array($2 * 2^n$). All amplitude 0 except index 0 = 1.
apply_gate	state, n, q, g	---- single-qubit gate application ---- Apply a 2×2 gate to qubit q (0 = least significant) of an n-qubit state. The gate is given as 4 complex numbers (8 doubles): $g = [a_{re}, a_{im}, b_{re}, b_{im}, c_{re}, c_{im}, d_{re}, d_{im}]$ representing $[[a,b],[c,d]]$.
qbit	k, mask	extract bit: $(k \text{ AND } mask) \neq 0 ? mask : 0$, via floor division returns 0.0 if bit clear, mask value if set
gate_h	g	Hadamard: $1/\sqrt{2} * [[1,1],[1,-1]]$
gate_x	g	Pauli-X (NOT): $[[0,1],[1,0]]$
gate_z	g	Pauli-Z: $[[1,0],[0,-1]]$
gate_y	g	Pauli-Y: $[[0,-i],[i,0]]$
gate_s	g	Phase gate S: $[[1,0],[0,i]]$
cnot	state, n, c, t	---- two-qubit CNOT ---- control qubit c, target qubit t: flip target when control = 1.
prob	state, k	probability of measuring basis state $k = amplitude_k ^2$
measure	state, n	measure the full register: returns a basis-state index (0..dim-1) sampled according to $ amplitude ^2$. Uses built-in random().
total_prob	state, n	total probability (should stay ~ 1.0 — a sanity/normalization check)

lib crypto

Teori bilangan & kriptografi edukasi — 17 fungsi

Fungsi	Parameter	Deskripsi
gcd	a, b	greatest common divisor (Euclid)
lcm	a, b	least common multiple
mod	a, m	modulo that works for our doubles: $a \bmod m$
modpow	base, exp, m	---- modular exponentiation: $(base^{exp}) \bmod m$ ---- square-and-multiply; keeps numbers small so it stays exact as long as $m*m < 2^{53}$ (i.e. $m < \sim 94$ million).

Fungsi	Parameter	Deskripsi
modinv	a, m	---- modular inverse via extended Euclid: $a^{-1} \bmod m$ ----
is_prime	n	trial-division primality test (fine for small n)
miller_test	n, a	Miller-Rabin single-base test (base a). Returns 1 if n is a probable prime for this base, 0 if definitely composite.
is_prime_mr	n	Miller-Rabin with a few fixed bases (good for numbers < ~3 million)
next_prime	n	next prime $\geq n$
totient_pq	p, q	Euler's totient for a product of two primes p, q
dh_public	g, secret, p	a party's public value: $g^{\text{secret}} \bmod p$
dh_shared	other_public, secret, p	shared secret from the other party's public value
rsa_n	p, q	
rsa_choose_e	phi	find a small public exponent e coprime with phi (tries 3,5,17,257,65537)
rsa_d	e, phi	private exponent $d = e^{-1} \bmod \phi$
rsa_encrypt	m, e, n	encrypt / decrypt a single number ($m < n$)
rsa_decrypt	c, d, n	

lib mechanic

Rekayasa mesin & mekanika — 26 fungsi

Fungsi	Parameter	Deskripsi
torque	force, radius	torque from force and moment arm: $T = F \cdot r$
power_rot	torque_nm, omega	power from torque and angular velocity: $P = T \cdot \omega$ (W)
rpm_to_rads	rpm	convert RPM to angular velocity (rad/s): $\omega = 2\pi \cdot \text{rpm} / 60$
rads_to_rpm	omega	
power_from_rpm	torque_nm, rpm	power from torque and RPM directly: $P = T \cdot 2\pi \cdot N / 60$
gear_ratio	driven_teeth, driver_teeth	gear ratio = driven teeth / driver teeth
gear_output_speed	input_speed, ratio	output speed after a gear stage: $n_{\text{out}} = n_{\text{in}} / \text{ratio}$
gear_output_torque	input_torque, ratio	output torque after a gear stage: $T_{\text{out}} = T_{\text{in}} \cdot \text{ratio}$
stress	force, area	normal stress: $\sigma = F / A$ (Pa)
strain	delta_length, original_length	strain: $\epsilon = \Delta L / L_0$ (dimensionless)
youngs_modulus	stress_val, strain_val	Young's modulus: $E = \text{stress} / \text{strain}$
elongation	force, length, area, e_modulus	elongation from Hooke's law: $\Delta L = F \cdot L / (A \cdot E)$
beam_moment_center	load, length	max bending moment: $M = P \cdot L / 4$
beam_deflection_center	load, length, e_modulus, inertia	max deflection: $\Delta = P \cdot L^3 / (48 \cdot E \cdot I)$
inertia_rect	width, height	area moment of inertia for a rectangular section: $I = b \cdot h^3 / 12$
inertia_circle	diameter	area moment of inertia for a solid circular section: $I = \pi \cdot d^4 / 64$
polar_inertia	diameter	polar moment of inertia, solid circular shaft: $J = \pi \cdot d^4 / 32$
shear_stress	torque_nm, radius, polar_j	max shear stress in a shaft: $\tau = T \cdot r / J$
spring_force	k, displacement	Hooke's law force: $F = k \cdot x$
spring_energy	k, displacement	elastic potential energy in a spring: $E = 0.5 \cdot k \cdot x^2$
kinetic_energy	mass, velocity	kinetic energy: $E = 0.5 \cdot m \cdot v^2$

Fungsi	Parameter	Deskripsi
potential_energy	mass, height	potential energy: $E = m \cdot g \cdot h$ ($g = 9.80665$)
reynolds	density, velocity, diameter, viscosity	Reynolds number: $Re = \rho \cdot v \cdot D / \mu$
flow_rate	velocity, diameter	volumetric flow rate in a pipe: $Q = v \cdot \pi \cdot (D/2)^2$
mech_advantage	load, effort	mechanical advantage: $MA = \text{load} / \text{effort}$
efficiency	output_power, input_power	efficiency (%): $\text{output} / \text{input} \cdot 100$

lib electric

Elektro & elektronika — 25 fungsi

Fungsi	Parameter	Deskripsi
voltage	current, resistance	$V = I \cdot R$
current	voltage_v, resistance	$I = V / R$
resistance	voltage_v, current_a	$R = V / I$
power	voltage_v, current_a	$P = V \cdot I$
power_ir	current_a, resistance	$P = I^2 \cdot R$
power_vr	voltage_v, resistance	$P = V^2 / R$
series2	r1, r2	two resistors in series: $R = R1 + R2$
parallel2	r1, r2	two resistors in parallel: $R = R1 \cdot R2 / (R1 + R2)$
parallel_equal	r, n	n equal resistors in parallel: $R = r / n$
cap_energy	capacitance, voltage_v	energy stored in a capacitor: $E = 0.5 \cdot C \cdot V^2$
charge	capacitance, voltage_v	charge on a capacitor: $Q = C \cdot V$
cap_series2	c1, c2	capacitors in series: $C = C1 \cdot C2 / (C1 + C2)$
cap_parallel2	c1, c2	capacitors in parallel: $C = C1 + C2$
rc_tau	resistance, capacitance	RC time constant (seconds): $\tau = R \cdot C$
rl_tau	inductance, resistance	RL time constant: $\tau = L / R$
rc_charge	vs, t, resistance, capacitance	capacitor charging voltage at time t: $V(t) = V_s \cdot (1 - e^{-(t/RC)})$
rc_discharge	v0, t, resistance, capacitance	capacitor discharging voltage: $V(t) = V_0 \cdot e^{-(t/RC)}$
omega	freq	angular frequency: $\omega = 2 \cdot \pi \cdot f$
reactance_c	freq, capacitance	capacitive reactance: $X_c = 1 / (2 \cdot \pi \cdot f \cdot C)$
reactance_l	freq, inductance	inductive reactance: $X_l = 2 \cdot \pi \cdot f \cdot L$
resonance_lc	inductance, capacitance	LC resonant frequency: $f = 1 / (2 \cdot \pi \cdot \sqrt{L \cdot C})$
impedance_rlc	r, xl, xc	impedance magnitude of a series RLC: $Z = \sqrt{R^2 + (X_l - X_c)^2}$
voltage_divider	vin, r1, r2	voltage divider: $V_{out} = V_{in} \cdot R2 / (R1 + R2)$
led_resistor	v_supply, v_led, current_a	LED series resistor: $R = (V_{supply} - V_{led}) / I$
db_power	p2, p1	decibels (power ratio): $dB = 10 \cdot \log_{10}(P2/P1)$

lib chem

Kimia — 25 fungsi

Fungsi	Parameter	Deskripsi
avogadro	—	
moles_from_particles	n_particles	moles from number of particles: $n = N / N_A$
particles_from_moles	moles	number of particles from moles: $N = n * N_A$
moles	mass, molar_mass	moles from mass and molar mass: $n = m / M$
mass_from_moles	moles_n, molar_mass	mass from moles and molar mass: $m = n * M$
R_gas	—	
gas_pressure	n, temp_k, volume	pressure (Pa) from n, T (K), V (m^3)
gas_volume	n, temp_k, pressure	volume (m^3) from n, T (K), P (Pa)
gas_moles	pressure, volume, temp_k	moles from P, V, T
gas_temp	pressure, volume, n	temperature (K) from P, V, n
combined_v2	p1, v1, t1, p2, t2	combined gas law: solve for V2 given $P_1V_1/T_1 = P_2V_2/T_2$
molarity	moles_n, volume_liters	molarity: $M = \text{moles} / \text{volume_liters}$
moles_from_molarity	molarity_m, volume_liters	moles of solute from molarity and volume
dilution_v1	c2, v2, c1	dilution: $C_1*V_1 = C_2*V_2$, solve for V1 (volume of stock needed)
mass_percent	mass_solute, mass_solution	mass percent: $(\text{mass_solute} / \text{mass_solution}) * 100$
ppm	mass_solute, mass_solution	parts per million: $(\text{mass_solute} / \text{mass_solution}) * 1e6$
ph	h_concentration	pH from hydrogen ion concentration: $pH = -\log_{10}[H^+]$
h_from_ph	ph_val	$[H^+]$ from pH: $[H^+] = 10^{(-pH)}$
poh	ph_val	pOH from pH: $pOH = 14 - pH$
oh_from_poh	poh_val	$[OH^-]$ from pOH
heat	mass, specific_heat, delta_temp	heat energy: $q = m * c * \Delta T$ (c = specific heat J/g/K)
delta_temp	heat_q, mass, specific_heat	solve for delta T given q, m, c
arrhenius	a_factor, ea, temp_k	rate constant: $k = A * \exp(-E_a/(R*T))$ E_a in J/mol, T in K
percent_yield	actual, theoretical	percent yield: $(\text{actual} / \text{theoretical}) * 100$
density	mass, volume	density: $\rho = \text{mass} / \text{volume}$

Total 255 fungsi di 15 library. Semua bisa ditumpuk dalam satu program dengan beberapa baris lib. Untuk membuat library sendiri, lihat MEMBUAT_LIBRARY.md atau Bab 12 pada Mike_Manual.pdf.

safetest-dev • Mike v1.0